

Final Project: Machine Learning Applications

Carla Aranda Sánchez: 100523031

David Gao: 100524564

Marina Juzgado Gómez-Menor: 100523023

Iván López Anca: 100523112

INTRODUCTION

Nowadays we are experiencing a huge development in the digital area, especially with the massive generation of text around the internet and the development of AI that uses this text as a fuel to exist and function. However, most of the text that is published is unstructured and a machine cannot work with plain text. Artificial Intelligence uses modern NLP (Natural Language Processing) to use that information published in the internet and train itself to understand context and perform different tasks.

NLP is a specialized branch of Machine Learning that provides the necessary tools to transform human language into numerical data that algorithms can process. These techniques are applied in many different fields, such as customer support through chatbots, monitoring brand reputation and performing Sentiment Analysis. Before having the commodity of the current AI models, NLP used to be work with different grammar rules, without learning and updating itself. Now, NLP learns through examples and finds patterns.

The following project consists on a processing of reviews. In a platform like Steam it is crucial to know the positive and negative feedback given to a certain game. Reading and classifying thousands of reviews is a task that consumes too much time for a human; however, as the developer of a game needs to make decisions based on the player experience, NLP is used to process those reviews, saving a lot of time. By classifying reviews into “Positive” or “Negative”, we transform raw text into a KPI. NLP is able to recognize if a review is negative because of the game or an external situation by analyzing whether topics relate to game or are unrelated complaints. At the same time, in the second task of the project, we would be able to know if the game has positive reviews because of the story, the graphics or the characters, and likewise for negative reviews.

The dataset was entirely created by the group, it is structured by 12,826 Steam reviews and 12 different variables distributed in columns. The reviews were obtained with a self-made code to scrape reviews from the Steam platform (store.steampowered.com) owned by Valve Corporation. The code extracts the raw text of the review and other useful variables such as the sentiment, language, or name of the game title.

TASK 1: TEXT PREPROCESSING AND VECTORIZATION

Step 1: Preprocessing pipeline

Before addressing any algorithm or processing, in every Machine Learning task it is important to first analyse the data we are working with. In the case of this project, it is first necessary to take a look at the dataset and the text reviews that will be processed. The initial insight shown after plotting different graphs is the number of words in each review; mostly we will be processing sequences between a very few and 200 characters. Moreover, taking into account the future binomial classification task that would classify reviews between “Positive” and “Negative”, we need to check the proportion of these reviews in the original dataset. Seeing that the proportion of positive reviews is higher than the negative ones, in the future Machine Learning task we would need to adjust the weights for a proper classification.

The raw data collected from Steam reviews presents several characteristics typical of user interaction and real-life language, which makes direct vectorization inadvisable. Reviews frequently contain HTML markups, URLs to external sites, emoji characters and non-ASCII symbols, contractions and colloquial words. To address these issues that are difficult for vectorization, a `wrangle_text` function is implemented in the first stage of the pipeline. Its logic is summarized below:

- **HTML removal:** parsed with BeautifulSoup to strip the markups while preserving the raw text.
- **URL removal:** detected and eliminated via the regular expression `r'https?://\S+|www.\S+'`.
- **Contractions expansion:** handled by the `contractions` library, which transforms abbreviated forms into their full words, ensuring the tokeniser receives full words.
- **Non-ASCII filtering:** emojis, symbols and accented characters from other languages are removed by encoding to ASCII and ignoring unrecognized bytes.
- **Lowercasing and whitespace normalization:** applied to make the text uniform and guarantee a consistent surface form.

After a language inspection, it was confirmed that all reviews in the dataset are written in English; therefore, the multilingual detection stage is omitted in order to reduce processing time.

Linguistic Processing with spaCy: The following step after wrangling is to configure the Natural Language Processing

engine using spaCy. To complete this task we load the model `en_core_web_md`, the medium-sized model that includes pre-trained vectors required for later embedding steps while remaining computationally feasible for a corpus of approximately 13,000 reviews. Disabling the parser and the NER speeds up the process, since neither syntactic trees nor entity spans are required for vectorization. Additional stopwords are added to the list; these are words that frequently appear in a videogame context and do not add predictive value for sentiment analysis. For each token produced by the tokeniser, the following filters are applied in order:

- 1) **Alphabetical check:** tokens containing digits or punctuation are discarded.
- 2) **POS filtering:** only tokens tagged as NOUN, PROPN, VERB, or ADJ are retained. Function words are excluded as they carry little semantic content.
- 3) **Stop-word removal:** words contained in spaCy’s stop word list and those previously classified as stopwords are removed.
- 4) **Lemmatisation:** each token is reduced to its dictionary base form so that all variants collapse into a single entry. Lemmas shorter than two characters are discarded.

A representative example of the transformation is shown below:

Raw:

“I don’t recommend this game. The graphics are stunning but the story feels rushed and the developers stopped updating it.”

After wrangling:

“i do not recommend this game the graphics are stunning but the story feel rushed and the developer stop update it”

After spaCy pipeline (lemmas):

```
['recommend', 'graphic', 'stun',
 'story', 'feel', 'rush', 'developer',
 'stop', 'update']
```

Step 2: Text Vectorization

1) Classical approaches: BoW and TF-IDF:

Once each review had been reduced to a list of lemmas, a Gensim dictionary was built to construct the vocabulary by mapping every unique lemma to an integer identifier. As the raw vocabulary contains approximately 16,000 unique terms, it was filtered to remove uninformative extremes:

- Terms appearing in **fewer than 5 documents** — these may be due to misspellings or highly specific proper nouns.
- Terms appearing in **more than 85% of documents** — these are practically universal words that do not add discriminative power for the analysis.

The filtering process caused a reduction of 76% in the vocabulary with respect to the raw one. The new compact vocabulary is highly optimized, reducing computational cost while retaining the most meaningful information. The top-20 most frequent terms extracted into the new vocabulary express different kinds of information useful for sentiment analysis. Mainly, words that

indicate strong sentiments dominate the ranking, while words that express topics in the videogame domain and user experience also add context and structure.

Bag-of-Words Representation (BoW): The Bag-of-Words (BoW) model represents each review as a sparse vector of length equal to the vocabulary size, where each dimension records the raw count of the corresponding term. The BoW vector for document d is:

$$\mathbf{x}_d^{\text{BoW}} = (c_{d,1}, c_{d,2}, \dots, c_{d,|V|})$$

where $c_{d,j}$ is the number of occurrences of term j in document d , and $|V|$ is the vocabulary size. Whereas BoW is simple and interpretable, it assigns equal importance to all terms regardless of their discriminative power across the corpus.

Term Frequency–Inverse Document Frequency (TF-IDF):

TF-IDF addresses the limitation of BoW by downweighting terms that appear frequently across many documents. The weight assigned to term j in document d is computed as:

$$x_{d,j}^{\text{TF-IDF}} = \text{freq}_{d,j} \times \log_2 \left(\frac{N}{n_j} \right)$$

where $\text{freq}_{d,j}$ is the relative frequency of term j in document d , N is the total number of documents, and n_j is the number of documents containing term j .

Terms that appear in many documents receive lower IDF weights, while terms that are rare in the corpus but frequent in a specific review receive a higher weight. This makes TF-IDF well-suited for sentiment classification, where opinion-bearing adjectives and domain-specific verbs tend to be the most discriminative features. The density analysis in Figure 1 reveals that the TF-IDF matrix has a mean density of 0.38%, meaning that over 99.6% of the matrix consists of zeros. This mathematically validates the decision to use Compressed Sparse Column matrices, optimizing memory consumption and accelerating training time in future Machine Learning algorithms.

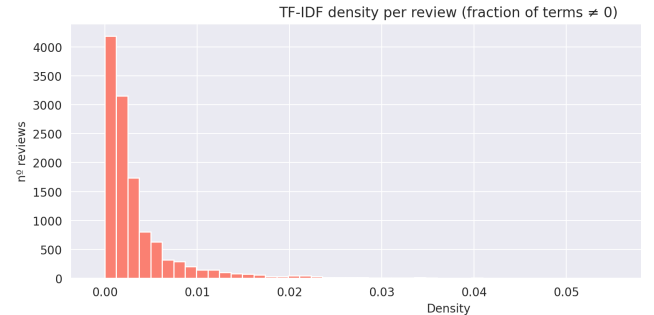


Figure 1: TF-IDF density distribution across the document corpus.

2) **Embedding-based approaches:** BoW and TF-IDF represent text as sparse, high-dimensional vectors and treat each word individually. They fail to capture semantic relationships; for example, models do not know that “fun” and “enjoyable” share similar meanings.

To address this drawback of the classical vectorization approaches, we implemented Word2Vec, an embedding-based approach that learns distributed representations of words in a dense and lower-dimensional space. Words that appear in similar contexts are converted into vectors located close to each other geometrically, capturing their meaning.

We decided to train a Word2Vec model on our Steam reviews since gaming terminology is highly specific; words like “bug”, “nerf” or “boss” have entirely different semantic meanings in a videogame context compared to standard English. This is the main reason why we did not choose GloVe, since it relies on global co-occurrence matrices, which requires massive training data, and using a pre-trained model would not take into account the specific vocabulary of the Steam reviews. We also did not consider FastText because its main advantage is handling typos, an issue we had already taken care of during preprocessing. Since our data is clean, Word2Vec is faster, simpler, and perfectly suited for our needs.

Training

We trained our custom Word2Vec model using the Skip-gram architecture with 200 dimensions to accurately capture semantic relationships. To optimize memory usage, we saved only the essential word vectors and deleted the heavy training model from RAM, which speeds up the rest of the process. We performed a quick similarity test to ensure the embeddings made sense. The results were highly accurate for the gaming domain: the word “boss” was correctly associated with “fight”, “tough”, and “enemy”, while “bug” was linked to “glitch” and “minor”. This confirms that our model successfully learned the specific vocabulary and context of the Steam reviews.

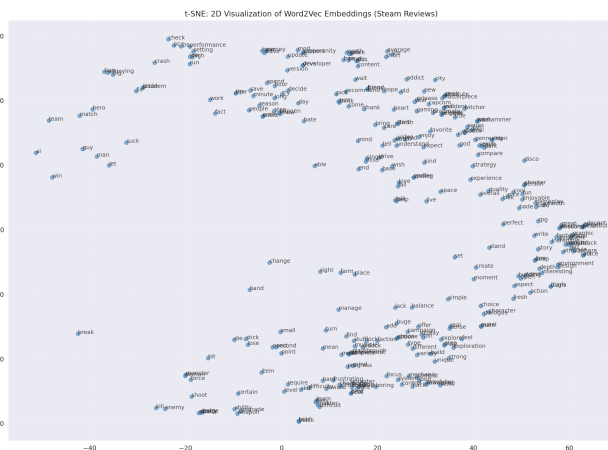


Figure 2: t-SNE 2D visualization of Word2Vec

Results and Visualization

To interpret the learned multidimensional relationships, we applied t-SNE to project our 200-dimensional vectors into a 2D space. Plotting the 300 most frequent words revealed clear “semantic islands” where related terms naturally cluster. It also highlighted gamer-specific contexts, such as linking the word “bug” directly to titles like ARK or Evolve. This confirms the main advantage of dense representations over sparse TF-IDF matrices: Word2Vec successfully captures the “gamer” language, recognizing that different synonyms often discuss the exact same underlying concept.

3) Topic modeling-based representation - Theme extraction and vector representation using the LDA:

We explore an alternative approach to text representation using Topic Modeling. Latent Dirichlet Allocation (LDA) models documents probabilistically, assuming each review is a mixture of underlying topics, and each topic is a collection of words. By using LDA, we can represent each review as a vector of topic probabilities, defining a document by how much it belongs to each theme. This provides a high-level, structural feature set for future classification tasks.

N-gram detection

Before training the LDA model, we performed N-gram detection using Gensim. This is crucial as gaming terminology relies strongly on multi-word phrases; treating words like “elden” and “ring” separately loses their semantic meaning. This method links frequently co-occurring terms into single concepts, preserving their context. The algorithm successfully identified 656 unique phrases, correctly grouping game titles (elden_ring, resident_evil), mechanics (boss_fight, open_world), and catchphrases (rip_tear). Treating these as single units ensures our LDA model discovers more coherent and interpretable topics.

Model Training and Advanced Filtering

After preparing the vocabulary with N-grams, we proceeded to train the LDA model using the tomotopy library. We set $k = 10$ as our baseline number of topics and trained the model for 1000 iterations to ensure stable convergence. To guarantee the highest quality in the extracted topics, we applied internal filtering during the model’s initialization. First, we set a minimum frequency (min_df=5) to ignore extremely rare words. More importantly, we used the rm_top=20 parameter to automatically remove the 20 most frequent words across the entire corpus. This is a highly effective technique to eliminate generic, dataset-wide terms (such as “game” or “play”) that might have survived the initial stopword removal, ensuring that only the true underlying semantic themes emerge.

Topic Interpretation

Once the training was completed, we extracted the top 10 words for each of the 10 latent topics to evaluate their meaning. The results were coherent, proving that the model successfully isolated specific gaming themes. For instance:

- **Topic 1** represents hardcore Souls-like mechanics (soul, hard, elden_ring).
- **Topic 2** clusters grand strategy and faction management (faction, total_war, warhammer).
- **Topic 4** is dedicated to survival horror remakes (original, remake, resident_evil).
- **Topic 6** groups open-world RPG narratives (quest, witcher, cyberpunk).

Interactive 2D Visualization (pyLDavis)

To visualize our model, we generated an interactive 2D map using pyLDavis. The projection confirms a robust semantic structure with excellent separation between gaming genres. Furthermore, the topic prevalence is remarkably balanced, ensuring that Machine Learning classifiers will not be biased toward a single dominant theme. Logically related themes, such as Story-rich RPGs and Narrative games, cluster near each other without overlapping.

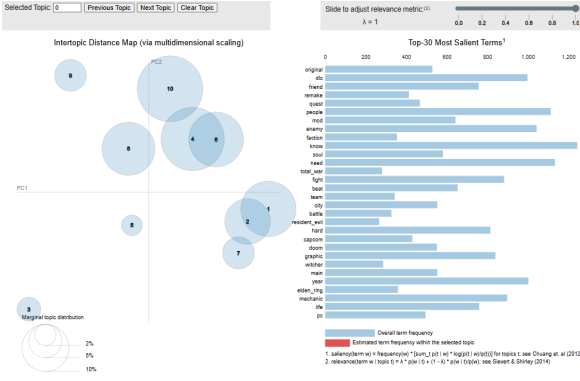


Figure 3: pyLDavis representation

Hyperparameter Tuning and Stability Analysis

While initial results were interpretable, we mathematically justified the number of topics by training models with $k \in [5, 10, 15, 20, 25, 50]$. We evaluated them using C_v Coherence (semantic similarity) and Jaccard Similarity (topic stability).

The analysis confirms $k = 10$ as optimal: it reached a high C_v coherence (0.5966) while maintaining low Jaccard redundancy. Beyond $k = 10$, coherence dropped and topics became redundant copies of existing ones, validating $k = 10$ as the ideal partitioning.

Semantic Validation and Final Feature Extraction

To demonstrate the model’s practical utility, we analyzed the cosine similarity between individual reviews. The model successfully distinguished a review complaining about the community (Topic 8) from one praising challenging mechanics (Topic 1), proving it accurately translates real-world semantic differences into mathematical distances. Additionally, the topic distribution showed a well-balanced corpus, ensuring our classification models will not suffer from severe thematic bias.

Finally, we performed the final vectorization. Every review was transformed into a 10-dimensional vector representing its topic probability distribution. This resulted in our final 12365×10 feature matrix (X_{topics}), a dense numerical representation that successfully captures the latent meaning of each text. This matrix was exported and will serve as the primary input for the Machine Learning classifiers in Task 2.

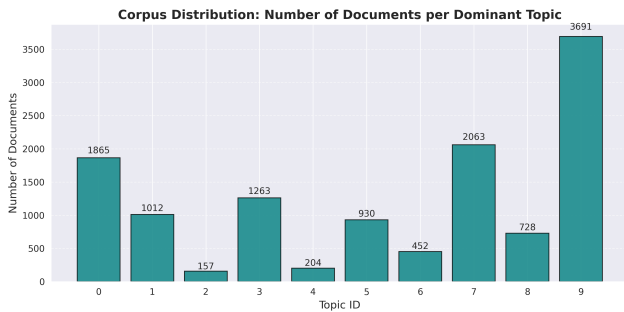


Figure 4: Corpus topic distribution

4) Contextual embedding:

Traditional vectorisation methods (BoW/TF-IDF, Word2Vec, LDA) all produce *static, context-free* representations. They cannot distinguish the word *bug* in a software context from *bug* as an

insect. BERT solves this through self-attention, dynamically computing each token’s embedding based on its full surrounding context, making it far better suited for sentiment-rich text like Steam reviews where meaning is heavily context-dependent.

Selected model: all-MiniLM-L6-v2

We selected sentence-transformers/all-MiniLM-L6-v2, a distilled version of BERT fine-tuned specifically for semantic sentence similarity via contrastive learning. Its key properties are:

- **Architecture:** 6-layer MiniLM transformer, distilled from a 12-layer BERT-base, retaining $\approx 95\%$ of performance at a fraction of the computational cost.
- **Output:** A single fixed-size 384-dimensional dense vector per document, directly usable as input to any downstream ML model.
- **Training objective:** Minimises cosine distance between semantically similar sentence pairs using NLI and paraphrase datasets.
- **Why not full BERT?** bert-base-uncased produces 768-dimensional vectors and is approximately $3\times$ slower to encode. Given our corpus of 12 826 reviews, MiniLM provides the best trade-off between quality and compute.
- **Normalisation:** All output vectors are L_2 -normalised (normalize_embeddings=True), so cosine similarity equals the dot product, making distance comparisons faster and numerically stable.

Encoding Pipeline

The raw review texts were fed directly to the model, since Transformer tokenisers handle punctuation, capitalisation, and subword splitting internally. Encoding was performed in batches of 64 reviews, producing a dense feature matrix X_{bert} of shape $12,826 \times 384$. The GPU was used when available; on the CPU the encoding takes approximately 5–10 minutes. The maximum sequence length of all-MiniLM-L6-v2 is 512 tokens. Given that the 75th percentile of review length in our corpus is approximately 62 tokens (after preprocessing), truncation affects a negligible fraction of documents.

Embedding Validation

Before using X_{bert} in Task 2, we validated that the model captured meaningful semantic structure through two complementary analyses: (i) pairwise cosine similarity checks on hand-picked review pairs and a random sample, and (ii) PCA projections coloured by sentiment and by game.

Looking at Figure 4, we see that the two distributions overlap almost entirely, with only a modest gap between the intra-class mean (0.273) and the inter-class mean (0.267). The near-zero difference between intra- and inter-class cosine means confirms that BERT embeddings do not form sentiment-pure clusters in cosine space. Sentiment polarity is distributed across the full 384-dimensional geometry rather than encoded as local similarity, which explains why a margin-maximising linear classifier (SVM) is necessary to exploit it

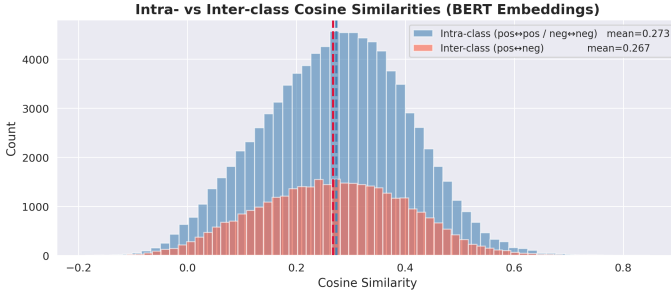


Figure 5: Pairwise cosine similarity distributions

The PCA plot in Figure 5 is not a strong endorsement of the embeddings for sentiment classification. The two classes occupy the same dense cloud with no visible boundary; negative reviews (red) scatter throughout the positive cluster (green) without forming their own region. It means that the raw BERT embeddings do not neatly separate sentiment in geometric space.

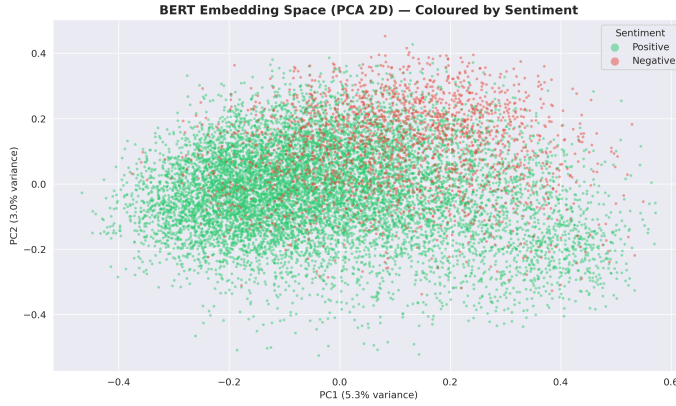


Figure 6: PCA projection of BERT embeddings

Limitations

- **Computational cost:** Encoding 12,365 reviews with a GPU takes 15–30 seconds; on a CPU it can take several minutes, making BERT less practical for real-time inference.
- **Max token length:** all-MiniLM-L6-v2 enforces a hard 512-token limit. Reviews exceeding this are silently truncated. Given our corpus has a mean length of ~ 62 tokens after preprocessing, truncation affects very few documents.
- **Black-box representations:** Unlike LDA topics or TF-IDF weights, the 384 BERT dimensions have no direct human-interpretable meaning, limiting qualitative analysis.

TASK 2: MACHINE LEARNING MODEL

A. Classification: Sentiment Prediction on Steam Reviews

Problem and Methodology: We address binary sentiment classification (positive / negative) of 12 358 Steam reviews (after

cleaning) using the four document representations built in Task 1: BoW, TF-IDF (4 376 terms), Word2Vec mean-pooled, LDA topic distributions and BERT sentence embeddings via all-MiniLM-L6-v2. The dataset is strongly imbalanced ($\approx 84\%$ positive / 16% negative, ratio 5.4:1), so all classifiers are trained with `class_weight='balanced'` and the primary metrics are F1-macro and ROC-AUC rather than accuracy.

Validation protocol: All comparisons use 5-fold Stratified Cross-Validation on the full corpus to preserve class proportions across folds. The best configuration is then re-tuned via GridSearchCV with the same CV scheme and finally evaluated on a held-out 20% stratified test split that is never seen during model selection.

Feature Extraction / Selection Analysis:

Three dimensionality-reduction strategies were applied with Logistic Regression as a fixed probe to isolate the effect of the representation: TruncatedSVD (LSA) on TF-IDF, SelectKBest (mutual information) on TF-IDF, and PCA on BERT embeddings.

Table I: Dimensionality reduction results (5-fold CV F1-macro).

Method	Best size	F1-macro	Baseline
TF-IDF + TruncatedSVD	500	0.7302	0.7534 (4 376)
TF-IDF + SelectKBest (MI)	3 000	0.7530	0.7534 (4 376)
BERT + PCA	200	0.7521	0.7585 (384)

Both TF-IDF reductions trade a small amount of F1 for a lighter feature space; SelectKBest is essentially loss-free at $k = 3,000$ (0.7530 vs. 0.7534) and is preferred for interpretability. PCA on BERT loses only 0.009 on F1 while halving the dimension, which is useful when memory or latency matter. Crucially, no projection beats its full baseline: the ceiling is set by the representation, not by feature count.

Model Comparison Across Vectorisations:

We ran a full grid (8 representations $\times$ 3 classifiers: Logistic Regression, calibrated Linear SVM, and Random Forest) under the same 5-fold Stratified CV.

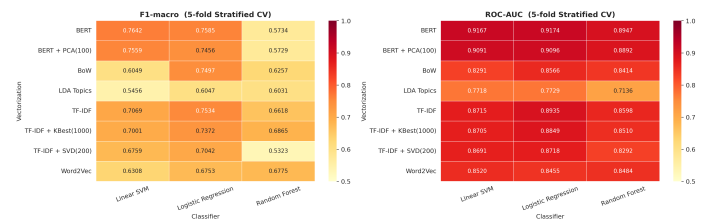


Figure 7: F1-macro and ROC-AUC across all representation-classifier combinations (5-fold CV).

The ranking is stable across both metrics. BERT dominates ($F1 \approx 0.76$, $AUC \approx 0.92$), followed closely by TF-IDF with linear models ($F1 \approx 0.75$); the BERT-vs-TF-IDF gap is only ≈ 0.01 F1, suggesting Steam sentiment is largely lexical (“fun”, “recommend”, “broken”). Word2Vec mean-pooling and LDA topics lag behind: averaging word vectors blurs polarity, and topics capture themes rather than polarity. Random Forest is consistently the worst choice on these high-dimensional / sparse representations, while linear classifiers excel.

Hyperparameter Tuning and Final Evaluation:

The winning combination BERT + Linear SVM was tuned with GridSearchCV over $C \in \{0.01, 0.1, 1, 10, 50\}$ and $\text{max_iter} \in \{1000, 2000\}$. Best parameters: $C = 0.1$, $\text{max_iter} = 1000$, raising CV F1-macro from 0.7642 to 0.7702. The model was retrained on 80% of the data and evaluated on the held-out 20%:

Table II: Test-set classification report — BERT + Linear SVM.

Class	Precision	Recall	F1	Support
Negative	0.67	0.49	0.56	386
Positive	0.91	0.95	0.93	2087
Accuracy			0.88	2473
Macro avg	0.79	0.72	0.75	2473
Weighted avg	0.87	0.88	0.87	2473

Why does BERT outperform bag-of-words methods? TF-IDF and BoW treat each word as an independent dimension and ignore word order. BERT encodes the full sentence context through a transformer architecture with 384-dimensional representations, capturing negation, intensifiers, and multi-word expressions that single-word frequencies miss.

Threshold tuning: The default 0.5 cut-off gave negative-class precision = 0.67. Maximising F1 on the precision–recall curve yields an optimal threshold of 0.395, which lifts negative precision to 0.76 (recall drops to 0.40). This is the right trade-off when false alarms on negative reviews are costlier than missed ones.

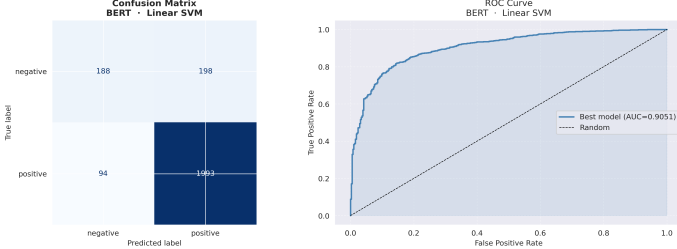


Figure 8: Precision–recall curve for BERT + Linear SVM. The optimal threshold (0.395) is highlighted.

Conclusions:

The best model, BERT + Linear SVM, achieves a held-out F1-macro of 0.75 and ROC-AUC of 0.917. Despite this, classical TF-IDF with a linear classifier remains within 0.01 F1 and is the strongest interpretable alternative, particularly after MI-based feature selection ($k = 3,000$), which recovers 98% of full-vocabulary performance while discarding 30% of terms. Feature-extraction techniques (TruncatedSVD, SelectKBest, PCA on BERT) consistently fall short of their full baselines, confirming that performance ceilings are set by representation quality, not feature count, though reductions to 100–200 components recover 97–99% of the score at a fraction of the computational cost.

The main bottleneck across all configurations is the 5.4:1 class imbalance. Even with balanced class weights, stratified CV, and threshold tuning, the negative-class recall reaches only 0.49, meaning roughly half of negative reviews are missed. Resolving this would require collecting more negative examples, applying synthetic oversampling in the embedding space, or incorporating

auxiliary signals such as playtime or helpfulness votes. Linear classifiers consistently outperform Random Forest on both dense and sparse inputs, and fine-tuning the BERT encoder on Steam-domain data remains the most promising avenue for further improvement.

B. Recommender Systems

Data Preparation:

The raw dataset does not contain user identifiers. Since recommender systems require them, synthetic users were created by hashing the first 50 characters of each review text, producing 3,000 pseudo-users. Duplicate (user, game) pairs were removed, retaining the interaction with the highest playtime to avoid noise from repeated entries.

Two rating schemes were derived from the raw data. A **binary rating** based on `voted_up` indicates whether the user liked the game and is used for content-based profile building. A **playtime-based rating** on a 1–5 scale is obtained by quintile-binning `playtime_at_review_hrs`, ensuring that each rating group contains the same number of observations and correcting the strong positive skew of raw playtime values

Content-Based Filtering:

The content-based approach represents each game as the centroid of the document vectors of all its reviews. Two games are considered similar if their centroid vectors are close in the embedding space. For a given user whose liked games form the set $\mathcal{L}$, the recommendation score for an unseen game i is:

$$s(u, i) = \frac{1}{|\mathcal{L}|} \sum_{j \in \mathcal{L}} \cos(\mathbf{c}_i, \mathbf{c}_j)$$

where $\mathbf{c}_k$ is the L2-normalised centroid of game k . For TF-IDF, a TruncatedSVD reduction to 200 dense dimensions is applied before computing centroids. The top- N unseen games ranked by $s(u, i)$ are returned.

We evaluate with **leave-one-out Hit-Rate@5**: for each user with at least two liked games, one liked game is held out and the system must recover it within the top-5 recommendations. This pipeline is applied to all four vectorisations from Task 1.

The best model is **BERT**, as it achieves the highest Hit-Rate, being the model that identifies best the context due to their contextual embedding systems. It knows how to classify games where users value the most “difficulty and mastery” (Elder Ring) and keeping them clearly apart from other type of games like “competitive meta- based ones (Dota 2)

It is followed by **TF-IDF + SVD**, which is a strong and interpretable runner-up. It groups specific vocabulary such as "dungeon", "ranked", or "craft" and concentrates them in the reduced latent space. Therefore, it separates game styles effectively, even it does not have contextual understanding.

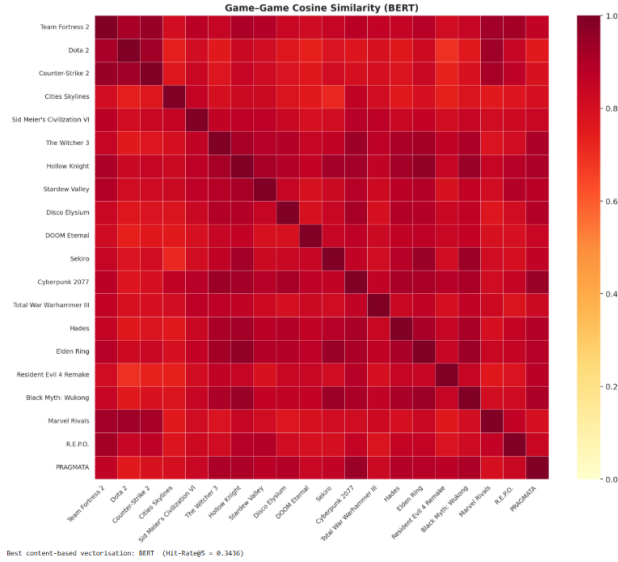


Figure 9: Game-game cosine similarity matrix for the best-performing vectorisation (BERT)

Word2Vec mean-pooling understands the general context at capturing semantic proximity between games, but it loses sentence structure and polarity. Consequently, its precision goes down when reviews contain mixed sentiment

LDA Topics is the weakest method. Its 10-dimensional topic distributions conflate theme with sentiment. It means that two games reviewed positively of unrelated genres can seem similar to LDA because both attract approving language

A shared limitation of all content-based methods here is the small item space of only 20 games, which caps recommendation diversity and inflates hit-rate variance. A broader catalogue would be expected to amplify the BERT advantage.

Collaborative Filtering:

Collaborative filtering exploits the rating matrix (`userID`, `gameID`, `playtime_rating`) without any text features, discovering patterns by crossing user interaction data. All models were implemented with the Surprise library and hyperparameters were selected by 3-fold GridSearchCV minimising RMSE.

BaselineOnly predicts a rating as $\hat{r}_{ui} = \mu + b_u + b_i$, all estimated by alternating least squares. It might look like a simple model. Nevertheless, it is a competitive baseline, because a large share of the rating variance in this dataset comes from systematic user leniency and per-game quality differences rather than personalised taste.

Three **KNN variants** were explored — KNNBasic, KNNWithMeans, and KNNWithZScore — searching over $k \in \{10, 20, 40\}$, $\text{min_k} \in \{3, 6, 10\}$, and similarity measures (MSD, cosine, Pearson), both user-based and item-based.

SVD (biased ALS) was tuned over $n_factors \in \{20, 50, 100\}$, $n_epochs \in \{20, 40\}$, learning rate $\in \{0.005, 0.01\}$, regularisation $\in \{0.02, 0.1\}$, and both biased and unbiased variants. **NMF** was also evaluated with a comparable grid.

SVD is the winner as it archives the lowest RMSE, meaning it fails less. The user and item bias terms are critical: they absorb the fact that some users always rate high and some games are

universally well-received, freeing the latent factors to model only the residual personalised deviations.

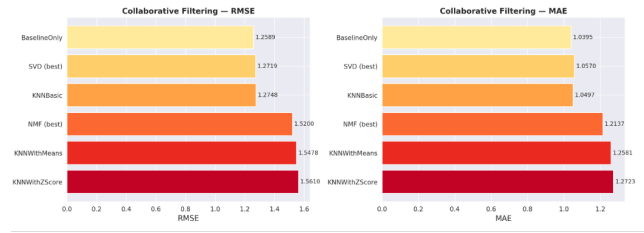


Figure 10: RMSE and MAE for all collaborative filtering models

KNNWithZScore outperforms KNNWithMeans and KNNBasic. As High-playtime users tend to assign top ratings to nearly every game, making their raw ratings uninformative; z-score normalisation is applied, which means that the system adjusts that to see the preferred games.

Item-based KNN works extremely better than **user-based KNN** in this dataset. With only 20 games but up to 3,000 users, item similarities are estimated from many interactions and are therefore stable, whereas user similarities are computed from a handful of ratings each and are noisy

NMF underperforms SVD despite a comparable search grid. Its non-negativity constraint prevents the model from representing negative deviations from average, for example a user who genuinely dislikes action games relative to their own mean.

BaselineOnly remains competitive throughout, despite its simplicity, which suggests that most of the rating signal comes from global biases rather than personalised taste.

Comparison and Conclusions: The two families of recommender systems are complementary rather than competing. Content-based filtering requires only a single review to produce recommendations, being therefore perfect for breaking the ice, while collaborative filtering leverages cross-user patterns to surface serendipitous discoveries that similarity-based methods cannot find.

The best content-based method is BERT with cosine similarity over game centroids, particularly effective for games with a distinctive and consistent review vocabulary. The best collaborative filtering model is SVD with biased ALS, which captures personalised latent preferences beyond what global biases alone can explain.

The recommended production architecture is a hybrid system: default to BERT content-based recommendations when a user has fewer than two interactions, and switch to SVD collaborative filtering once sufficient history is available. This is the standard approach adopted by major streaming and gaming platforms, and it directly addresses the two main failure modes — cold start for content-based and sparsity for collaborative filtering.

The primary limitation of this study is the catalogue size of 20 games. This constrains the collaborative methods, inflates the relative strength of the baseline, and caps content-based diversity. A larger item space would be expected to amplify the SVD advantage over KNN methods and widen the performance gap between BERT and the simpler vectorisations.

FINAL COMPARISON: ALL VECTORIZATIONS ACROSS ALL TASKS

This section consolidates results from Task 1 (intrinsic properties) and Task 2 (downstream performance) into a unified view.

Task 1 — Intrinsic Properties

Table III: Intrinsic properties of each vectorization.

Vectorization	Dims	Sparsity	Vocab coverage	Special metric
BoW	4,376	0.9962	4,376 terms	—
TF-IDF	4,376	0.9962	4,376 terms	Mean density: 0.38%
Word2Vec	200	0.0013	93.2% tokens	Token coverage: 0.9325
LDA Topics	10	0.0000	—	C_v coherence ($k = 10$): 0.5927
BERT	384	0.0000	subword tokenizer	Intra-Inter gap: 0.0467

BERT intra-class similarity: 0.2844 Inter-class: 0.2377 Separation gap: 0.0467

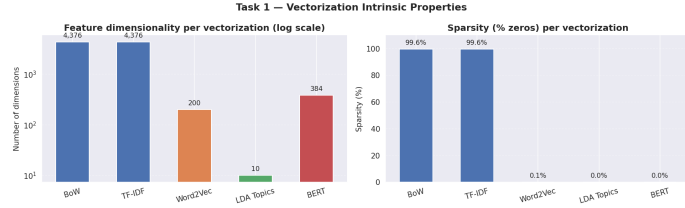


Figure 11: Dimensionality (log scale) and sparsity per vectorization.

Full Comparison — Tasks 1, 2.1 and 2.3

Table IV: Combined performance across all tasks.

Vectorization	Best F1-macro	Best ROC-AUC	Hit-Rate@5	Sparsity
BERT	0.7642	0.9174	0.3350	0.0000
BERT + PCA(100)	0.7559	0.9096	—	—
TF-IDF	0.7534	0.8935	—	—
BoW	0.7497	0.8566	—	0.9962
TF-IDF + KBest(1000)	0.7372	0.8849	—	—
TF-IDF + SVD(200)	0.7042	0.8718	0.3307	0.9962
Word2Vec	0.6781	0.8504	0.3295	0.0013
LDA Topics	0.6047	0.7729	0.2908	0.0000

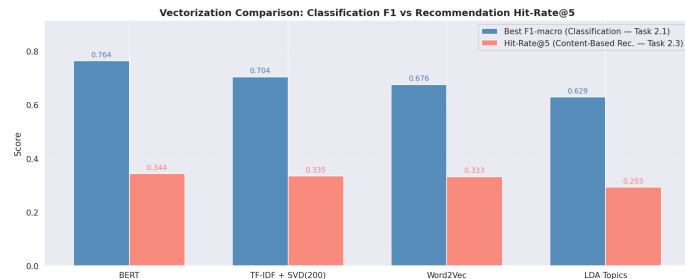


Figure 12: Classification F1-macro vs. content-based Hit-Rate@5 per vectorization.

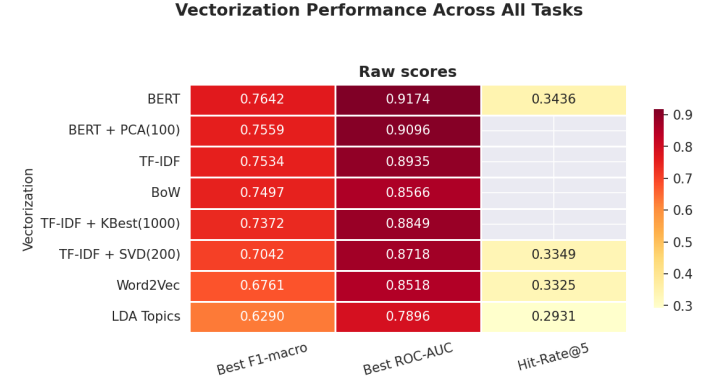


Figure 13: Raw scores and relative rank per metric across all vectorizations.

Final Winners

- **Classification (Task 2.1):** BERT + Linear SVM, F1-macro = 0.7642, ROC-AUC = 0.9167.
- **Content-based (Task 2.3):** BERT, Hit-Rate@5 = 0.3350.
- **Collaborative filtering (Task 2.3):** BaselineOnly, RMSE = 1.2703, MAE = 1.0478.
- **Most compact:** LDA Topics (10 dims, dense); highest dimensionality: BoW/TF-IDF (4,376 dims, 99.6% sparse).

The results obtained across all tasks reveal a consistent and coherent hierarchy among the five vectorization strategies evaluated. BERT contextual embeddings are the best-performing representation, achieving the highest F1-macro (0.7642) in sentiment classification, the highest ROC-AUC (0.9167), and the best Hit-Rate@5 (0.3350) in content-based recommendation, confirming that having the word-order and contextual meaning means measurable gains across both discriminative and retrieval tasks. Nevertheless, the margin over classical methods is narrower than the expected: full TF-IDF reaches an F1-macro of 0.7534 (only 0.011 points below BERT), which suggests that Steam sentiment is largely keyword-driven and that a well-tuned sparse representation can compete with a 384-dimensional contextual model at a fraction of the computational cost.

The dimensionality reduction experiments reinforce this point, as applying PCA to BERT (transforming into 100 components) or SelectKBest to TF-IDF (1,000 features) degrades performance only marginally, indicating that the relevant signal is concentrated in a small subspace of each representation. Word2Vec occupies a middle ground: its dense 200-dimensional embeddings capture game gender specific semantics but lose document-level structure during mean-pooling, placing it below TF-IDF in classification. LDA Topics, proves to be the weakest representation for both tasks, due to the fact that it is designed to deal with thematic content rather than sentiment polarity.

The intrinsic properties further illuminate these trade-offs: BoW and TF-IDF are extremely sparse (99.62 percent zeros), however, this sparsity is handled efficiently with compressed sparse matrices. On the other hand, BERT and LDA are fully dense but of very different scales (384 vs. 10 dimensions). On the collaborative filtering side, the unexpected strong performance of BaselineOnly shows a limitation of the experimental setup: When using only 20 items and creating users synthetically,

neighbourhood and latent-factor models have not enough signal to outperform a simple bias-correction model.

This means that we cannot make any conclusions about SVD or KNN superiority, because it is a special case for the dataset we have. Taken together, we can state that for applications where interpretability and speed matter, TF-IDF with a linear classifier offers a compelling trade-off; while for applications where semantic richness is the priority, BERT-based representations should be used, justifying their higher computational cost.

A natural next step would be a hybrid architecture that combines BERT game centroids for content-based cold-start scenarios with a latent-factor collaborative model once sufficient user-item interaction data has accumulated, thereby exploiting the complementary strengths identified throughout this project.

ACKNOWLEDGMENT OF AUTHORSHIP

The dataset used in this project was built entirely by the group using a self-developed Python web scraping script that collected publicly available user reviews from the Steam platform (*store.steampowered.com*), owned by Valve Corporation. A total of 12,826 reviews were gathered, along with associated metadata such as sentiment label, language, and game title. No third-party datasets or pre-collected corpora were used. All remaining code, analysis, and written content in this report are original work produced by the authors.